

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра програмної інженерії

Звіт
З дисципліни «Аналіз та рефакторинг коду»
З лабораторної роботи №4

Виконав:
ст. гр. ПЗП-22-9

Терновський Денис

Перевірив:

Дашенков Д. С.

Харків 2024

4 РОЗРОБКА ІоТ КЛІЄНТА (БІЗНЕС-ЛОГІКИ ТА ФУНКЦІЙ НАЛАШТУВАННЯ)

Мета:

Розробити програмне забезпечення для IoT або SmartDevice пристрою, створеного на базі будь-якої поширеної на сьогодні платформи, придатної для реалізації вбудованих систем (Embedded System).

Хід роботи:

Основні компоненти проєкту TaskLamp

1. ESP32 DevKit V1

- Основний мікроконтролер, який відповідає за управління компонентами пристрою, збір даних, синхронізацію часу, обробку завдань і комунікацію з сервером через WiFi.

2. OLED-дисплей (128x64, SSD1306)

- Відображає поточний час або список завдань. У разі відсутності завдань виводить повідомлення "**0 tasks found**".
- Використовується для швидкого перегляду стану системи.

3. LED кільце (WS2812B)

- RGB-кільце з 16 світлодіодами для індикації.
- Світло змінює кольори та яскравість залежно від налаштувань і стану пристрою.

4. Потенціометри

- **Яскравість (BRIGHTNESS_PIN):** Регулює яскравість LED кільця (0-255).
- **Колір (COLOR_PIN):** Встановлює колір LED кільця, базуючись на значеннях аналогового входу.

5. RTC-модуль (DS1307)

- Реальний час синхронізується для відображення поточного часу на екрані.

6. WiFi-з'єднання

- Використовується для обміну даними із сервером:
 - Отримання списку завдань.
 - Синхронізація параметрів і оновлення статусу.

7. Програмне забезпечення

- Логіка керування системою забезпечує:
 - Перемикання між режимами (відображення часу та завдань).
 - Синхронізацію даних із сервером.
 - Управління світлом через кольори та яскравість.

Принцип роботи системи TaskLamp

1. Ініціалізація системи:

- ESP32 підключається до WiFi, ініціалізує дисплей, LED кільце та RTC.
- Завантажується список завдань із сервера.
- На OLED-дисплеї відображається початкова інформація.

2. Відображення часу та завдань:

- Система перемикається між показом часу та списком завдань кожні 5 секунд.

- Якщо завдання відсутні, виводиться текст "**0 tasks found**".

3. Обробка даних потенціометрів:

- Параметри кольору та яскравості зчитуються з потенціометрів.
- Користувач може змінювати колір світла залежно від настрою або вимог.

4. Комунікація із сервером:

- Дані про завдання отримуються через HTTP-запит.
- Завдання оновлюються, коли система перемикається в режим відображення списку завдань.

5. LED індикація:

- Кольори змінюються залежно від положення потенціометра.
- Світло адаптується до рівня яскравості, встановленого користувачем.

6. Реакція на зміни:

- У разі втрати WiFi-з'єднання або інших проблем із сервером система продовжує відображати останні отримані дані.
- При повторному з'єднанні завдання оновлюються автоматично.

Ця система об'єднує функціональність розумного світильника з можливістю відображення завдань, що забезпечує зручність і персоналізацію для користувача.

Підключення компонентів до ESP32:

Компонент	Опис
ESP32 DevKit V1	Основний мікроконтролер, який керує всіма компонентами пристрою, синхронізує дані з сервером через WiFi.
OLED-дисплей (SSD1306)	Відображає час, список завдань або повідомлення про їх відсутність. Використовується для візуалізації інформації.
LED кільце (WS2812B)	RGB-кільце на 16 світлодіодів, яке змінює кольори та яскравість для створення відповідного освітлення.
RTC-модуль (DS1307)	Синхронізує реальний час для точного відображення на OLED-дисплеї.
Потенціометр (BRIGHTNESS_PIN)	Регулює яскравість світлодіодів у діапазоні від 0 до 255.
Потенціометр (COLOR_PIN)	Змінює кольори RGB-кільця відповідно до заданого значення.
WiFi-з'єднання	Забезпечує комунікацію з сервером для завантаження завдань і синхронізації даних.
HTTPClient	Бібліотека для здійснення HTTP-запитів на сервер, отримання та обробки даних.
Adafruit SSD1306	Бібліотека для роботи з OLED-дисплеєм (ініціалізація, відображення тексту та графіки).
Adafruit NeoPixel	Бібліотека для управління RGB-кільцем (зміна кольорів, яскравості, ефектів).

Опис роботи системи TaskLamp

1. Ініціалізація системи:

- Після ввімкнення ESP32 підключається до WiFi-мережі, використовуючи задані параметри (SSID та пароль).
- Виконується ініціалізація OLED-дисплея, LED кільця та RTC-модуля.
- На OLED-дисплеї відображається повідомлення про успішний запуск системи.

2. Відображення інформації:

- **Час:**
 - На дисплеї відображається поточний час у форматі ГГ:XX:СС, оновлення кожену секунду.
- **Завдання:**

- Список завдань завантажується із сервера через HTTP-запит і відображається на дисплеї.

- Якщо завдань немає, виводиться повідомлення **"0 tasks found"**.

- Перемикання між режимами (відображення часу та завдань) здійснюється кожні 5 секунд.

3. Обробка завдань:

- Завантаження завдань із сервера відбувається під час переходу в режим відображення завдань.

- Дані завдань обробляються (парсер JSON) і зберігаються у локальному масиві.

4. Управління LED кільцем:

- **Яскравість:**

- Регулюється через потенціометр (BRIGHTNESS_PIN), значення конвертуються з діапазону 0-4095 у 0-255.

- **Колір:**

- Потенціометр (COLOR_PIN) задає RGB-компоненти:

- Червоний (R): пропорційно значенню 0-4095.

- Зелений (G): пропорційно до половини значення.

- Синій (B): пропорційно верхній половині значення.

- Кільце змінює колір та яскравість у режимі реального часу.

5. Синхронізація із сервером:

- Дані завдань завантажуються через HTTP-запит (GET).

- У разі успішного запиту завдання оновлюються. Якщо запит не вдається, виводиться відповідне повідомлення в серійну консоль.

6. Робота в реальному часі:

- Постійно оновлюються показники часу, завдань, кольорів і яскравості.

- Оновлення системи відбувається з затримкою 100 мс для забезпечення плавної роботи.

7. Реакція на помилки:

- Якщо WiFi-з'єднання відсутнє, в серійній консолі виводиться повідомлення про проблему.

- Якщо сервер недоступний або повертає код помилки (не 200), виводиться відповідне повідомлення про проблему.

8. Простота взаємодії:

- Система автоматично налаштовує всі компоненти, не потребуючи додаткових дій від користувача після запуску.

- Користувач може змінювати лише яскравість і колір LED кільця за допомогою потенціометрів.

Ця система поєднує функції відображення часу, списку завдань і керування освітленням у простому та зручному форматі.

Створити діаграму діяльності для IoT клієнта.

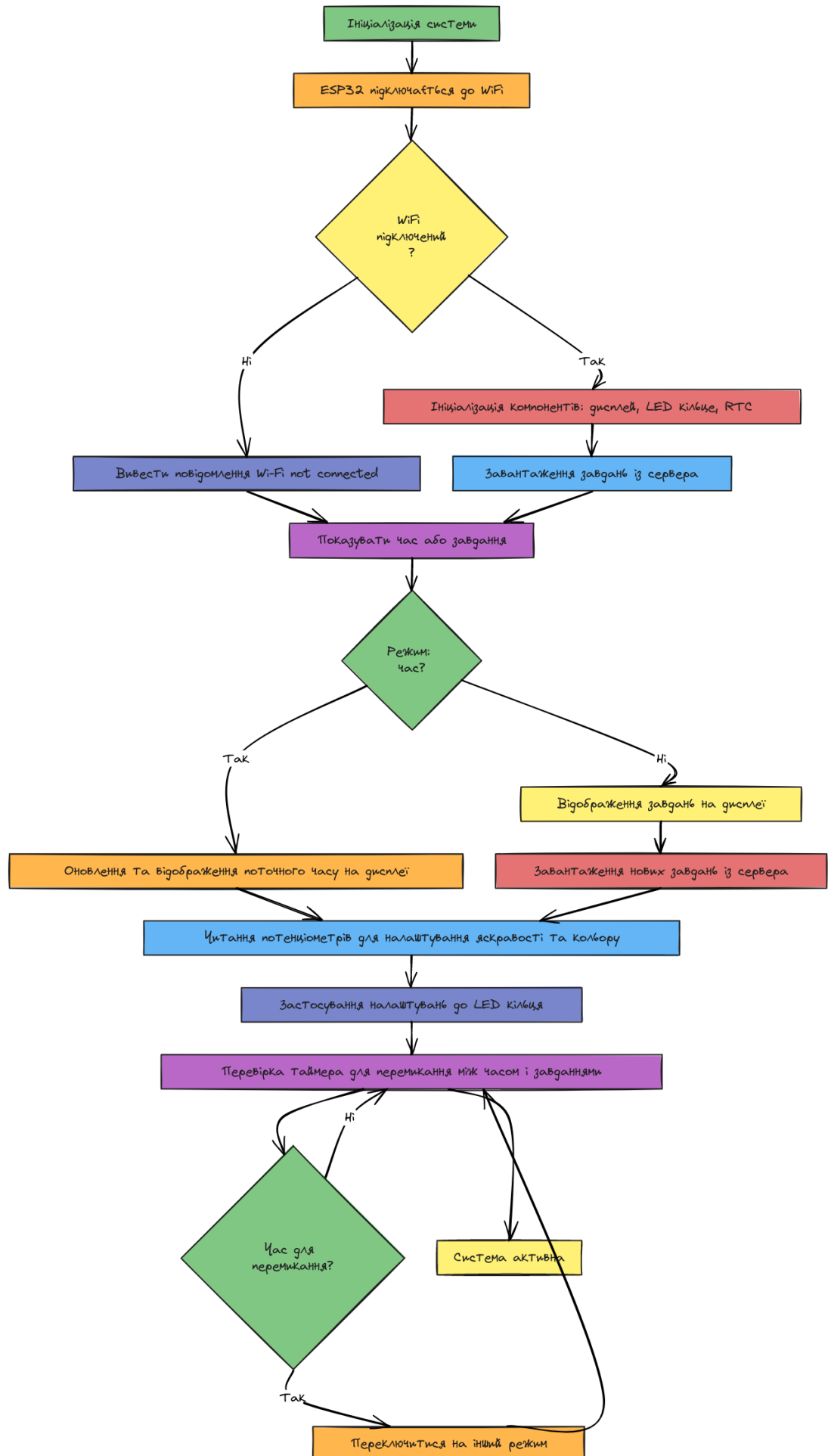


Рисунок 2 – Діаграма діяльності IoT- клієнта

1) Створити програмну реалізацію бізнес логіки та функцій налаштування IoT клієнта.

Для створення IoT-клієнта було використано додаток WOKWI. У якості мікроконтролера було обрано **ESP32 DevKit V1**, який забезпечує керування всіма компонентами та комунікацію із сервером через WiFi. Для відображення даних у реальному часі застосовано **OLED-дисплей SSD1306 (128x64)**, який показує поточний час та список завдань.

Керування освітленням реалізовано за допомогою **RGB LED кільця (WS2812B)**, яке змінює кольори та яскравість залежно від параметрів, заданих потенціометрами. Потенціометр **BRIGHTNESS_PIN** використовується для регулювання яскравості, а **COLOR_PIN** для зміни кольору.

Система також інтегрує **RTC-модуль (DS1307)** для забезпечення точного часу, який відображається на дисплеї. Комунікація із сервером реалізована через HTTP-запити для завантаження списку завдань і синхронізації даних.

Завдання з сервера оновлюються кожні 5 секунд або при переході до режиму їх відображення. У разі перевищення ліміту потужності або іншого аварійного стану система може інтегрувати зумер та світлодіоди для звукових та візуальних сповіщень.

Ця система забезпечує ефективне керування даними, надаючи користувачу інтуїтивно зрозумілий інтерфейс та адаптивну індикацію, що відповідає потребам сучасного IoT-рішення.

На рисунку 3 зображено схему пристрою, що використовується.

Посилання на схему: <https://wokwi.com/projects/419863947034949633>

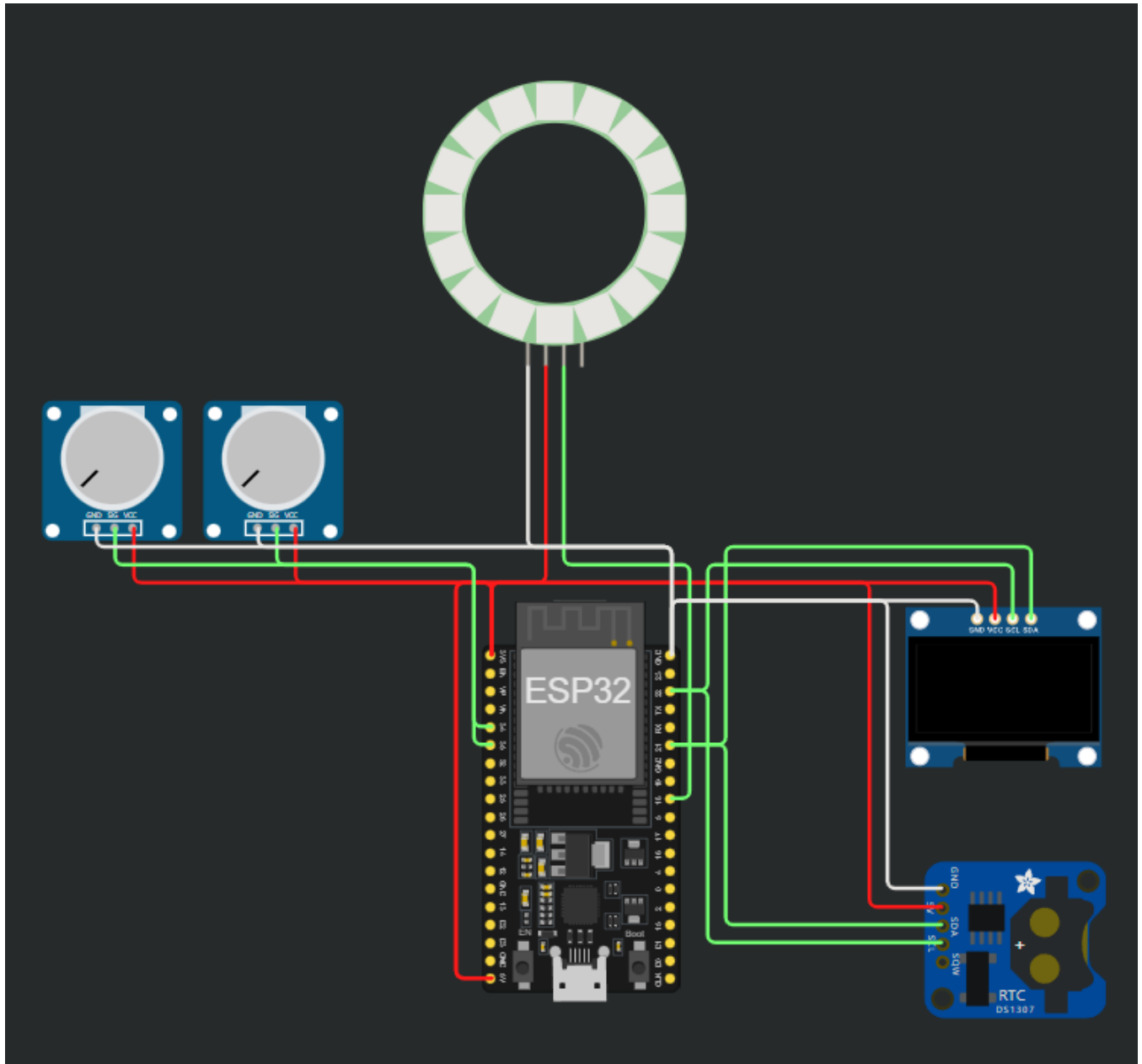


Рисунок 3 – Схема пристрою IoT- клієнта

Код налаштування IoT клієнта та програмної реалізації бізнес логіки представлений у додатку А.

2) Перевірити роботу IoT клієнта.

Для перевірки роботи IoT клієнта використаємо під'єднання до WiFi та перевіримо спочатку чи працюють датчики та чи правильно підраховуються дані. Далі спробуємо відправити ці дані на сервер.

1) Перевірка роботи датчиків і підрахунку даних:

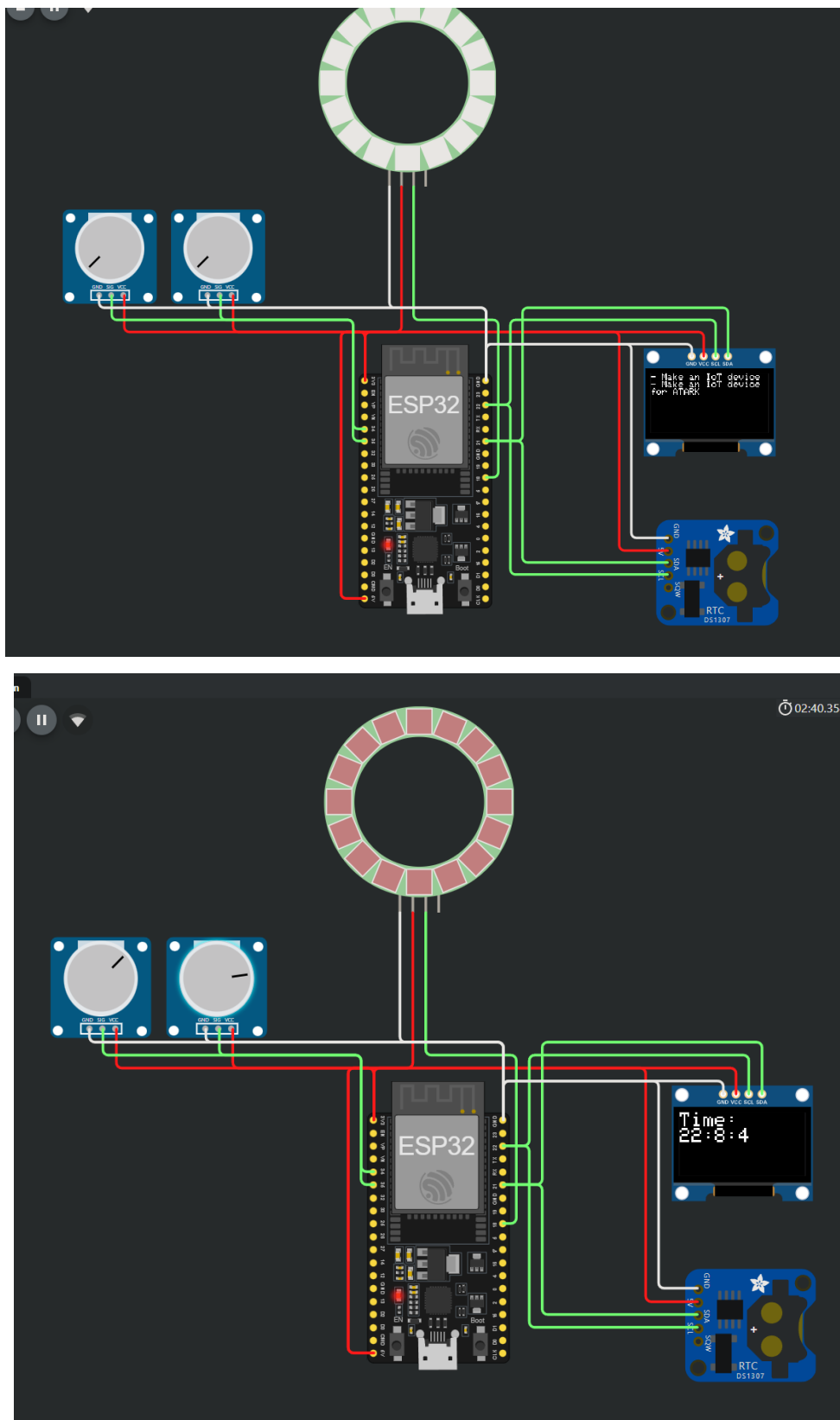


Рисунок 4 – Виведення даних з датчиків

Бачимо, що датчики працюють коректно і дані рахуються як треба.

2) Відправка даних на сервер:

```

ESP32
Hello, ESP32!
Connecting to Wi-Fi.
Connected to Wi-Fi

```

Рисунок 5 – Підключення до WiFi

```

// Завантаження завдань із сервера
void loadTasks() {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        http.begin(serverUrl);
        int httpResponseCode = http.GET();

        if (httpResponseCode == 200) {
            String payload = http.getString();
            parseTasks(payload);
        } else {
            Serial.println("Failed to fetch tasks, HTTP code: " + String(httpResponseCode));
        }
        http.end();
    } else {
        Serial.println("Wi-Fi not connected");
    }
}

// Розбір JSON із сервера
void parseTasks(String payload) {
    // Очищення попередніх завдань
    taskCount = 0;

    // Спрощений парсер JSON
    int startIdx = 0, endIdx = 0;
    while ((startIdx = payload.indexOf("\"title\":\"", endIdx)) >= 0) {
        endIdx = payload.indexOf("\"", startIdx + 9);
        if (endIdx > 0 && taskCount < MAX_TASKS) {
            tasks[taskCount++] = payload.substring(startIdx + 9, endIdx);
        }
    }
}

```

Рисунок 6 – Створення HTTP запиту і надсилання його на сервер

ДОДАТОК А

Код програми

Налаштування роботи IoT-пристрою

sketch.ino

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_NeoPixel.h>
#include <RTClib.h>

// Wi-Fi параметри
const char* ssid = "Wokwi-GUEST";
const char* password = "";
const char* serverUrl = "http://607f-109-86-145-249.ngrok-free.app/tasks/user/1"; //
URL сервера

// Параметри OLED дисплея
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Параметри LED кільця
#define LED_PIN 18 // GPIO18 для WS2812B
#define NUM_LEDS 16
Adafruit_NeoPixel strip = Adafruit_NeoPixel(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);

// Об'єкт RTC
RTC_DS1307 rtc;

// Піни потенціометрів
#define BRIGHTNESS_PIN 34 // Пін для яскравості
#define COLOR_PIN 35 // Пін для кольору

// Максимальна яскравість (0-255)
uint8_t brightness = 150;

// Масив для зберігання завдань
#define MAX_TASKS 10
String tasks[MAX_TASKS];
int taskCount = 0;

// Таймер для перемикання між часом і завданнями
unsigned long lastSwitchTime = 0;
const unsigned long switchInterval = 5000; // 5 секунд
bool showTime = true; // Показувати час чи завдання
```

```

void setup() {
    // Ініціалізація серійного монітора
    Serial.begin(115200);
    Serial.println("Hello, ESP32!");

    // Ініціалізація Wi-Fi
    WiFi.begin(ssid, password);
    Serial.print("Connecting to Wi-Fi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.print(".");
    }
    Serial.println("\nConnected to Wi-Fi");

    // Ініціалізація OLED дисплея
    if (!display.begin(SSD1306_BLACK, 0x3C)) {
        Serial.println(F("SSD1306 allocation failed"));
        for (;;)
            ;
    }
    display.clearDisplay();
    display.display();

    // Ініціалізація RTC
    if (!rtc.begin()) {
        Serial.println("Couldn't find RTC");
        for (;;)
            ;
    }
    if (!rtc.isrunning()) {
        Serial.println("RTC is NOT running, setting the time!");
        rtc.adjust(DateTime(F(__DATE__), F(__TIME__))); // Встановити час за компіляцією
    }

    // Ініціалізація LED кільця
    strip.begin();
    strip.show(); // Очистити кільце

    // Налаштування піна для потенціометрів
    pinMode(BRIGHTNESS_PIN, INPUT);
    pinMode(COLOR_PIN, INPUT);

    // Завантажити завдання
    loadTasks();
}

void loop() {
    // Оновлення часу
    DateTime now = rtc.now();

    // Кожні 5 секунд перемикається між часом і завданнями
    if (millis() - lastSwitchTime >= switchInterval) {
        lastSwitchTime = millis();
        showTime = !showTime;
    }
}

```

```

    // Завантажити завдання, якщо переходимо до їх відображення
    if (!showTime) {
        loadTasks(); // Отримати найсвіжіші завдання
    }
}

if (showTime) {
    displayTime(now);
} else {
    displayTasks();
}

// Читання значення потенціометрів
brightness = analogRead(BRIGHTNESS_PIN) / 16; // Перетворення 0-4095 у 0-255
uint16_t colorValue = analogRead(COLOR_PIN); // Значення кольору 0-4095

// Визначення кольору залежно від значення
uint8_t red = map(colorValue, 0, 4095, 0, 255); // Червоний
uint8_t green = map(colorValue, 0, 2047, 0, 255); // Зелений
uint8_t blue = map(colorValue, 2048, 4095, 0, 255); // Синій

// Застосування яскравості
strip.setBrightness(brightness);

// Встановлення кольору
setRingColor(strip.Color(red, green, blue));

delay(100); // Оновлення
}

// Функція для управління кольором LED кільця
void setRingColor(uint32_t color) {
    for (int i = 0; i < NUM_LEDS; i++) {
        strip.setPixelColor(i, color);
    }
    strip.show();
}

// Відображення часу на OLED
void displayTime(const DateTime& now) {
    String timeString = String(now.hour()) + ":" + String(now.minute()) + ":" +
String(now.second());
    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(0, 0);
    display.println("Time:");
    display.println(timeString);
    display.display();
}

// Відображення завдань на OLED
void displayTasks() {

```

```

display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(0, 0);
if (taskCount == 0) {
    display.println("0 tasks found");
} else {
    for (int i = 0; i < taskCount; i++) {
        display.println("- " + tasks[i]);
    }
}
display.display();
}

// Завантаження завдань із сервера
void loadTasks() {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        http.begin(serverUrl);
        int httpResponseCode = http.GET();

        if (httpResponseCode == 200) {
            String payload = http.getString();
            parseTasks(payload);
        } else {
            Serial.println("Failed to fetch tasks, HTTP code: " + String(httpResponseCode));
        }
        http.end();
    } else {
        Serial.println("Wi-Fi not connected");
    }
}

// Розбір JSON із сервера
void parseTasks(String payload) {
    // Очищення попередніх завдань
    taskCount = 0;

    // Спрощений парсер JSON
    int startIdx = 0, endIdx = 0;
    while ((startIdx = payload.indexOf("\"title\":\"", endIdx)) >= 0) {
        endIdx = payload.indexOf("\"", startIdx + 9);
        if (endIdx > 0 && taskCount < MAX_TASKS) {
            tasks[taskCount++] = payload.substring(startIdx + 9, endIdx);
        }
    }
}
}
}

```

diagram.json

```

{
  "version": 1,
  "author": "Егор Сотников",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": -4.76, "attrs": {} },
    {
      "type": "board-ssd1306",
      "id": "oled1",
      "top": 3.14,
      "left": 221.03,
      "attrs": { "i2cAddress": "0x3c" }
    },
    {
      "type": "wokwi-led-ring",
      "id": "ring1",
      "top": -277.76,
      "left": -37.21,
      "attrs": { "pixels": "16" }
    },
    { "type": "wokwi-ds1307", "id": "rtc1", "top": 138.6, "left": 230.5, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "pot1", "top": -106.9, "left": -153.8, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "pot2", "top": -106.9, "left": -240.2, "attrs": {} }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [ ] ],
    [ "esp:RX", "$serialMonitor:TX", "", [ ] ],
    [ "ring1:VCC", "esp:5V", "red", [ "v115.2", "h-48", "v211.2" ] ],
    [ "ring1:GND", "esp:GND.2", "black", [ "v96", "h76.8" ] ],
    [ "ring1:IN", "esp:D18", "blue", [ "v-70", "h10" ] ],
    [ "oled1:VCC", "esp:3V3", "red", [ "v-19.2", "h-268.65", "v38.4" ] ],
    [ "oled1:GND", "esp:GND", "black", [ "v10", "h5" ] ],
    [ "oled1:SCL", "esp:D22", "blue", [ "v15", "h10" ] ],
    [ "oled1:SDA", "esp:D21", "green", [ "v15", "h10" ] ],
    [ "rtc1:5V", "esp:5V", "red", [ "h-38.4", "v-173.2", "h-220.8", "v211.2" ] ],
    [ "rtc1:GND", "esp:GND", "black", [ "v25", "h10" ] ],
    [ "rtc1:SCL", "esp:D22", "blue", [ "v30", "h15" ] ],
    [ "rtc1:SDA", "esp:D21", "green", [ "v30", "h15" ] ],
    [ "oled1:SCL", "esp:22", "green", [ "v-28.8", "h-162.9", "v67.2" ] ],
    [ "oled1:SDA", "esp:21", "green", [ "v-38.4", "h-163.13", "v105.6" ] ],
    [ "oled1:GND", "esp:GND.2", "black", [ "v-9.6", "h-163.2" ] ],
    [ "ring1:DIN", "esp:18", "green", [ "v105.6", "h67.2", "v124.8" ] ],
    [ "rtc1:GND", "esp:GND.2", "black", [ "h-28.8", "v-153.6", "h-115.2" ] ],
    [ "rtc1:SDA", "esp:21", "green", [ "h-115.2", "v-96.3" ] ],
    [ "rtc1:SCL", "esp:22", "green", [ "h-124.8", "v-134.6" ] ],
    [ "pot1:VCC", "esp:3V3", "red", [ "v28.8", "h104.65" ] ],
    [ "pot2:VCC", "esp:3V3", "red", [ "v28.8", "h191.05" ] ],
    [ "pot1:GND", "esp:GND.2", "black", [ "v9.6", "h220.8" ] ],
    [ "pot1:SIG", "esp:34", "green", [ "v19.2", "h105.2", "v86.4" ] ],
  ]
}

```

```

    [ "pot2:SIG", "esp:35", "green", [ "v19.2", "h191.6", "v96" ] ],
    [ "pot2:GND", "esp:GND.2", "black", [ "v9.6", "h307.2" ] ]
  ],
  "dependencies": {}
}
}

```

Налаштування серверної частини

tasks.js

```

const express = require('express');
const router = express.Router();
const { Task } = require('../models');

// Отримати всі завдання
router.get('/', async (req, res) => {
  try {
    const tasks = await Task.findAll();
    res.json(tasks);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Отримати завдання за ID
router.get('/:id', async (req, res) => {
  try {
    const task = await Task.findByPk(req.params.id);
    if (!task) {
      return res.status(404).json({ message: 'Task not found' });
    }
    res.json(task);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Отримати всі завдання за UserID
router.get('/user/:userId', async (req, res) => {
  try {
    const tasks = await Task.findAll({
      where: {
        userId: req.params.userId,
      },
    });
    if (tasks.length === 0) {
      return res.status(404).json({ message: 'No tasks found for this user' });
    }
    res.json(tasks);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

```

```
// Створити нове завдання
router.post('/', async (req, res) => {
  try {
    const { userId, title, description, dueDate, isCompleted } = req.body;
    const newTask = await Task.create({ userId, title, description, dueDate,
isCompleted });
    res.status(201).json(newTask);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Оновити інформацію про завдання
router.put('/:id', async (req, res) => {
  try {
    const task = await Task.findByPk(req.params.id);
    if (!task) {
      return res.status(404).json({ message: 'Task not found' });
    }
    const { title, description, dueDate, isCompleted } = req.body;
    task.title = title || task.title;
    task.description = description || task.description;
    task.dueDate = dueDate || task.dueDate;
    task.isCompleted = isCompleted !== undefined ? isCompleted : task.isCompleted;
    await task.save();
    res.json(task);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Видалити завдання
router.delete('/:id', async (req, res) => {
  try {
    const task = await Task.findByPk(req.params.id);
    if (!task) {
      return res.status(404).json({ message: 'Task not found' });
    }
    await task.destroy();
    res.json({ message: 'Task deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Пошук завдань за назвою
router.get('/search/:title', async (req, res) => {
  try {
    const tasks = await Task.findAll({
      where: {
        title: req.params.title,
      },
    });
  });
});
```

```
    if (tasks.length === 0) {
      return res.status(404).json({ message: 'No tasks found' });
    }
    res.json(tasks);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Пошук завдань за статусом виконання
router.get('/status/:isCompleted', async (req, res) => {
  try {
    const isCompleted = req.params.isCompleted === 'true';
    const tasks = await Task.findAll({
      where: {
        isCompleted,
      },
    });
    res.json(tasks);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

module.exports = router;
```